

A WALKTHROUGH FOR STUDENTS

How an AI coding agent is actually built

Inside **llmc-code** — a Claude-Code-style assistant that runs entirely on your own laptop. We open every box: the loop, the tools, the memory, the search engine.

~13,000 lines of Python

26 modules

1,355 tests

3 dependencies

100% local

Every slide answers three questions — WHY it exists · WHAT it is · HOW it works

01 The problem we are solving

WHY THIS EXISTS

You can download a good model today. You still can't use it.

LM Studio will happily run a 30-billion-parameter coding model on your laptop. What you get is a **chat box**. What you wanted is something that reads your repo, edits files, runs your tests and fixes what broke.

THE GAP

A model can only turn text into text. It cannot open a file, run a command, remember yesterday, or stop itself when it is confused.

Something has to sit between the model and your computer. That something is a **harness**, and building one is what this deck is about.

Why not just use the cloud tools?

- **Privacy** — your source code never leaves the machine.
- **Cost** — no per-token bill, no rate limits.
- **It works offline** — on a plane, in a lab, behind an air gap.

BUT LOCAL CHANGES EVERYTHING

A cloud harness can assume a big, fast, well-behaved model. **We can assume none of that.** Every custom piece in the second half of this deck exists because of that one difference.

Four constraints — and what each one forced us to build

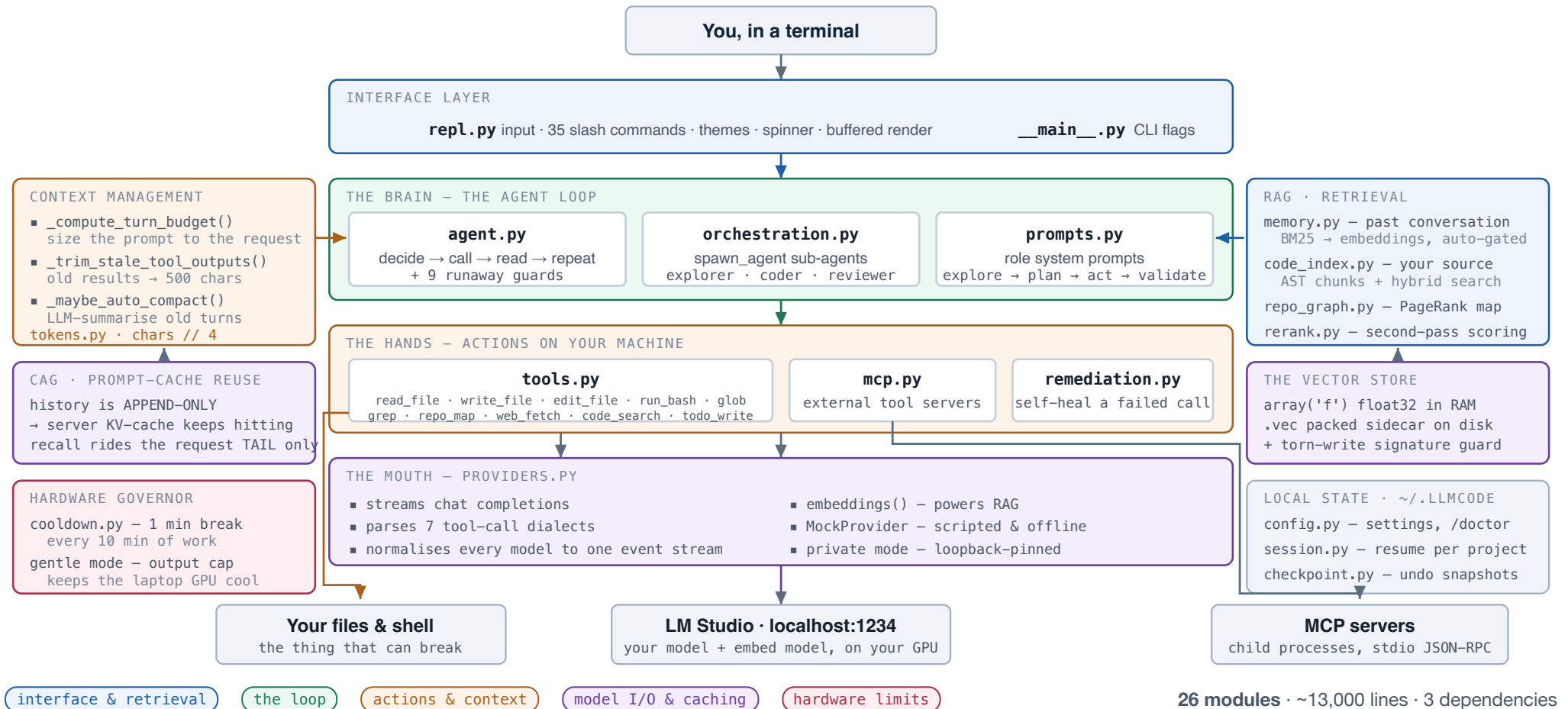
CONSTRAINT	WHAT WE HAD TO BUILD
Small, messy models no guaranteed tool-call format	7 tool-call parsers + an argument-alias layer, because every model family invents its own syntax
One laptop GPU prefill cost dominates	Context management + CAG: turn budgets, stale trimming, compaction, prompt-cache reuse
Small context windows and slow to fill	RAG + a vector store: search memory and code, send only what this question needs
Consumer hardware gets hot	Cooldown + gentle mode: forced breaks and output caps so a long session doesn't cook the machine
It edits real files	Confirmation gates, workspace confinement, undo checkpoints, an offline lockdown mode
3 dependencies only	BM25, cosine similarity, a vector store, PageRank and an MCP client — all hand-written

IN ONE LINE

A harness for running LLMs locally = the loop, the tools, and everything needed to make a small model on modest hardware behave like a useful engineer.

02 The complete architecture

EVERY MODULE, ONE PICTURE



WHY IT IS SHAPED LIKE THIS

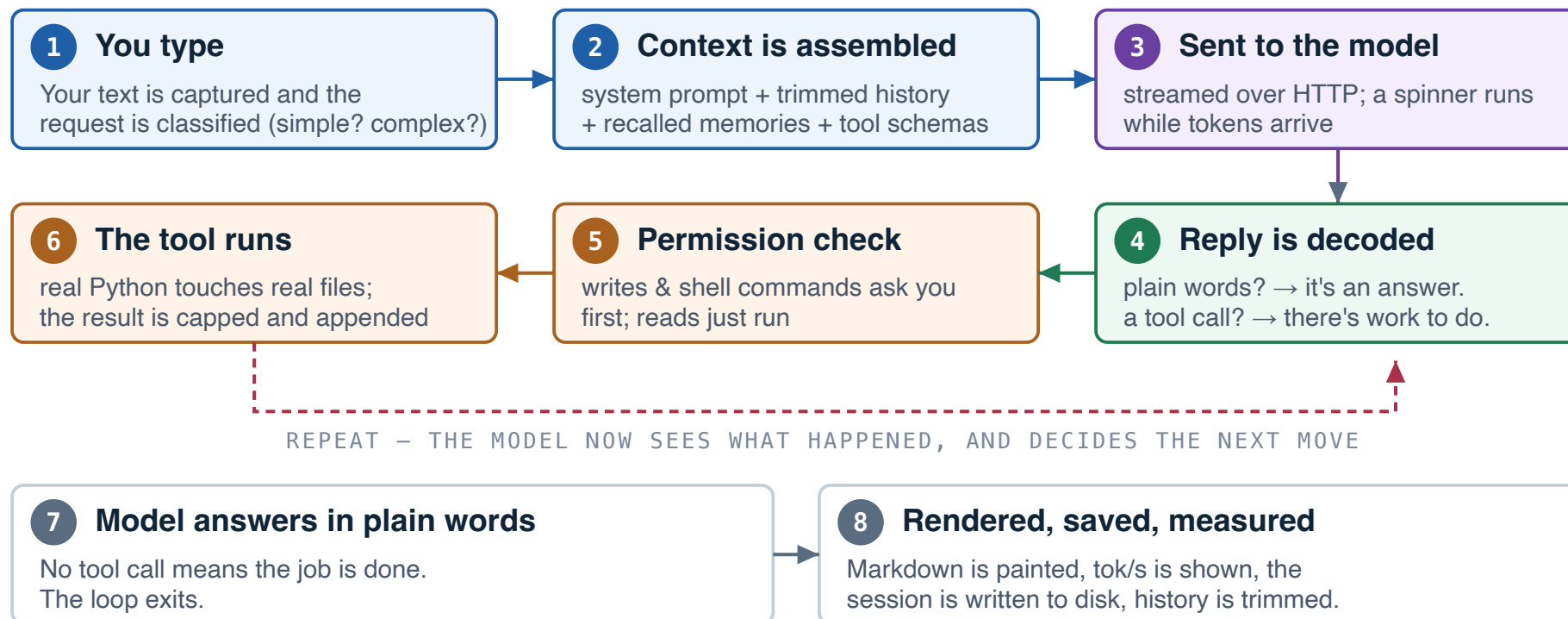
The **brain never touches your disk or the network**. It only asks the hands (tools.py) and the mouth (providers.py). That one rule makes the whole system testable without a GPU.

WHAT THE SIDE PANELS ARE

Left = **making it fast and safe on a laptop** (context, CAG, cooldown). Right = **making it knowledgeable** (RAG, vectors, local state). Neither exists in a cloud agent — they are our local-first additions.

HOW TO READ IT

Top-down is one request. Left and right are services the brain calls *before* and *around* each model call. The bottom row is the only part that is not our code.

**WHY A LOOP**

One question-and-answer round cannot fix a bug. The model must look, act, **see what happened**, then act again. Each pass it knows strictly more than the last.

WHAT ENDS IT

A reply with **no tool calls**. Plain words mean the model has nothing left to look up — so its words are the answer. The text is **buffered, then rendered once** as markdown, so headings and code fences display properly instead of as literal `##`.

HOW IT STAYS SAFE

A hard iteration cap (80), plus eight more guards. Writes and shell commands ask you first; reads just run.

04 The whole agent, in 20 lines

THE LOOP

Strip away 3,000 lines of production hardening and `Agent.run()` is this. **This is a real, working agent** — tested with a scripted fake model.

the core of every coding agent ever written

```
def agent(task, max_steps=12):
    messages = [{"role": "system", "content": SYSTEM},
                {"role": "user", "content": task}]

    for _ in range(max_steps):          # the iteration guard
        reply = client.chat.completions.create(
            model=MODEL, messages=messages, tools=SCHEMA
        ).choices[0].message
        messages.append(reply.model_dump(exclude_none=True))

        if not reply.tool_calls:         # plain words = DONE
            return reply.content

        for call in reply.tool_calls:
            args = json.loads(call.function.arguments)
            if call.function.name in NEEDS_OK and not ask_user():
                out = "DENIED by the user."
            else:
                try:
                    out = TOOLS[call.function.name](**args)
                except Exception as e:    # errors are DATA
                    out = f"ERROR: {type(e).__name__}: {e}"
            messages.append({"role": "tool",
                            "tool_call_id": call.id,
                            "content": str(out)[:4000]}) # ALWAYS cap

    return "Stopped: hit the step limit."
```

Five decisions hide in those lines

THE LINE	WHY IT MATTERS
<code>str(out)[:4000]</code>	An uncapped result poisons the context forever. Cap at the source .
<code>except → "ERROR: ..."</code>	A tool must never raise . The error is fed back so the model can correct itself.
<code>range(max_steps)</code>	Without it, one confused model loops all night.
<code>NEEDS_OK</code>	The human stays in the loop for anything destructive.
The system prompt	Behaviour is configured in English . "Explore before you answer" changes everything.

THE ERROR RULE IS THE BIG ONE

Compare two tool errors:

- "error" → the model guesses, and death-spirals.
- "path not found; did you mean llmcode/agent.py?" → fixed next turn.

Error messages are prompts. Write them for the model, not for a log file.

05 Tools — giving the model hands

TOOLS.PY

A TOOL IS THREE THINGS

llmcode/tools.py — the Tool dataclass

```
Tool(
    name="read_file",
    description="Read a text file. Use before answering about code.",
    parameters={"type": "object", "required": ["path"], # JSON Schema →
                "properties": {"path": {"type": "string"}, # sent to the model
                               "offset": {"type": "integer"},
                               "limit": {"type": "integer"}},
    fn=read_file, # the Python that runs
    requires_confirmation=False, # reads are safe
)
```

1. a schema (documentation *for the model*) · 2. a Python function · 3. a safety flag.

TOOL RESULTS HAVE ONE SHAPE

`{"ok": true, "result": ...}` or `{"ok": false, "error": ...}`. They **never** raise. A crash would end the turn; an error just teaches the model.

SCHEMA WORDING IS PROMPT ENGINEERING

"Read a file" gets you whole-file reads. "Read a file — use `offset / limit` for big files" gets you slices. Same code, different behaviour.

The eleven tools

TOOL	PURPOSE & THE GUARD ON IT
<code>read_file</code>	Slices via offset/limit; huge-file guard; skip-list
<code>write_file</code>	Confirm + workspace-confined
<code>edit_file</code>	Exact string replace, with a tolerant fallback
<code>run_bash</code>	Confirm, timeout, 256 KB output ceiling, secrets scrubbed from child env
<code>glob / grep</code>	Find files / find text — both result-capped
<code>repo_map</code>	Compact structural index of the repo
<code>code_search</code>	BM25 + embedding search over your code
<code>web_fetch</code>	SSRF-guarded : no loopback/private IPs, IP-pinned, re-checked per redirect
<code>spawn_agent</code>	Delegate to a scoped sub-agent
<code>todo_write</code>	The model's own checklist

TWO REAL SAFETY BUGS WE FIXED

- The docs claimed writes were "workspace-confined" — **they weren't**. A false security claim is worse than none.
- `write_file` corrupted files whose *content* contained tool-call markup. The parser swallowed the next call.

WHY

Cloud models are served with one strict tool-call format. Local models come from a dozen labs, each with its own chat template. Many **do not emit native tool calls at all** — they print the call as ordinary text and expect the client to notice.

WHAT

A parsing layer that recognises **one native format plus six text dialects** — DeepSeek, Qwen/Hermes, Mistral, Llama (two spellings), Qwen-XML, and a fenced-JSON protocol we invented for models that support nothing.

HOW

Detect the dialect by its marker → extract name and body → coerce to a Python dict. Everything downstream sees **one shape** and never learns a dialect existed.

THE BUG THAT FORCED THIS

`write_file` failed for weeks with *"requires a string 'path'"*. Two stacked causes: qwen emitted **XML parameters, not JSON**, so the native `arguments` field arrived empty — and it used **Claude's** parameter names.

THE HARD PART: WHERE DOES A VALUE END?

Models often open `<parameter=content>`, dump an entire Python file, and **never close the tag**. So a value must end at whichever terminator comes first. Overshoot and you write the *next tool call's markup* into the user's source file. That is exactly the write-corruption bug we fixed.

07 Custom #2 — wrong argument names, and refusing to guess

TOOLS.PY

Even once the call is extracted, the **names may be wrong**. Local models are trained on Claude's and OpenAI's docs, so they call our `path` parameter `file_path`, and our `old` parameter `old_string`.

llmcode/tools.py — the alias table (excerpt)

```
_ARG_ALIASES = {
  "read_file": {
    "path": ("file_path", "filepath", "filename", "file", "target_file"),
  },
  "write_file": {
    "path": ("file_path", "filepath", ...),
    # NOTE: deliberately NO new_string/new_str here — those are edit_file's
    # `new` key; mapping them to write_file content would turn a mis-targeted
    # edit into a DESTRUCTIVE whole-file overwrite with just the fragment.
    "content": ("file_text", "text", "body", "data", "code"),
  },
  "edit_file": {
    "old": ("old_string", "old_str", "search", "find"),
    "new": ("new_string", "new_str", "replace"),
  }
}
```

READ THAT COMMENT AGAIN

An alias table is a **guessing machine**. The author deliberately left out a plausible alias *because the failure mode was destructive*. Knowing which guess not to make is the whole skill.

The rule that makes aliasing safe

```
for canonical, syns in aliases.items():
    cur = args.get(canonical)
    if cur not in (None, ""):
        continue # model gave a real value — authoritative
    if cur == "":
        continue # explicit empty — intentional, keep it
    for syn in syns: # else backfill from the first alias present
        ...
```

- **An alias may only fill a blank** — never overrule what the model actually said.
- **Explicit empty wins.** `content: ""` means "create an empty file".
- **Copy-on-write** — so the confirmation prompt you saw and the arguments that ran can never drift apart.

And two places we deliberately stop

SITUATION	WHAT WE DO
Two tool-shaped blocks in one reply	Refuse to run any. Executing only the first silently drops the rest — the model would never learn its second instruction vanished.
A block inside prose or a <code><think></code> section	Ignore it. Otherwise the model <i>explaining</i> <code>rm -rf</code> would actually run it.

THE TRANSFERABLE RULE

Be liberal in what you accept, strict in what you execute. A missed call costs one turn. A wrong call costs your source file.

08 Custom #3 — why long chats kill a local model

WHY

Every turn re-sends the **entire** conversation. Answering has two phases: **prefill** (read the whole prompt — cost grows with length) and **decode** (write the reply — roughly constant). On a laptop GPU, prefill is where the time goes.

WHAT

A per-message context budget, plus machinery to keep history under it — so the model sees **what this message needs, and nothing else**.

HOW

Four levers, cheapest first: **measure** → **budget** → **trim** → **compact**.

ANALOGY

A colleague with **total amnesia between sentences**. To answer, they must re-read the entire meeting transcript — every time. Decode speed is how fast they talk. Prefill is how long they sit re-reading. Fifty pages of stale notes doesn't slow their speech; it makes them silent for two minutes first.

THE MEASUREMENT THAT STARTED THIS WORK

A user typed "*what happened?*" — four words. The program sent **55,000 tokens** of stale history. The reply took **~20 minutes**.

Auditing showed tool outputs were **56%** of the history; counting file content stored inside past `write_file` calls, over **90%**.

55k → 5k

tokens sent
for the same question

20m → 24s

wall clock
same model, same laptop

The four levers

LEVER	WHAT IT DOES	COST
Measure	<code>chars // 4</code> — no tokenizer dependency	microseconds
Budget	Size the prompt to the request (×1–3; trivial gets half)	microseconds
Trim	Old tool results → 500 chars, deterministically	milliseconds
Compact	Model-written summary of old turns	a whole model call

BIAS YOUR HEURISTICS SAFELY

The request classifier **deliberately errs upward**: when unsure, treat it as a real task. Misjudging a task as trivial **breaks the answer**; misjudging trivia as a task just wastes a little time.

TRIM — FREE, EVERY TURN

Old tool **results** cut to 500 chars; file content inside old tool **calls** cut to 160.

Three invariants: **idempotent** (a marker stops it re-cutting its own note), **structure-preserving** (ids stay paired), **actionable** (the note says how to get the rest back).

COMPACT — COSTS A MODEL CALL

The model summarises old turns into one note: decisions, files touched, open tasks.

Not truncation — deleting messages loses the decisions inside them.

COMPACTION CAN BACKFIRE

A weak model sometimes writes a summary as long as the original. So: **snapshot first**, require a **≥3% saving**, otherwise **roll back** and don't retry until history grows 10%. Rewriting history also costs the prompt cache — see the next slide.

WHY

The server keeps a **KV cache**: state for tokens it already read. If your next request starts with the **byte-identical** prefix, prefill for all of it is **skipped** — most of your waiting time, for free.

WHAT

One rule, enforced everywhere: **history is append-only**. Never insert, never reorder, never rewrite the front.
RAG makes prompts smarter; CAG makes them cheaper.

HOW

Anything dynamic — recalled memories, per-turn hints — is appended to a **copy** of the list, at the very end, on the first call only. It rides the uncached tail, which costs nothing.

THE SAME DISCIPLINE, THREE MORE PLACES

- Text-only messages keep `content` a plain **string** (attaching an image makes it a list — a different prefix).
- Tool schemas are **memoised**, so key order can't drift.
- A useless compaction is **rolled back** — the code would rather carry extra tokens than break the cache.

WHY THE RECALL NOTE IS NEVER STORED

Keeping it out of `self.messages` wins three times: the prefix stays cacheable, it is **never written to the session file** (so it can't leak), and compaction can never mangle it — because it was never there.

11 Custom #6 — RAG: remembering without re-reading

MEMORY.PY

WHY

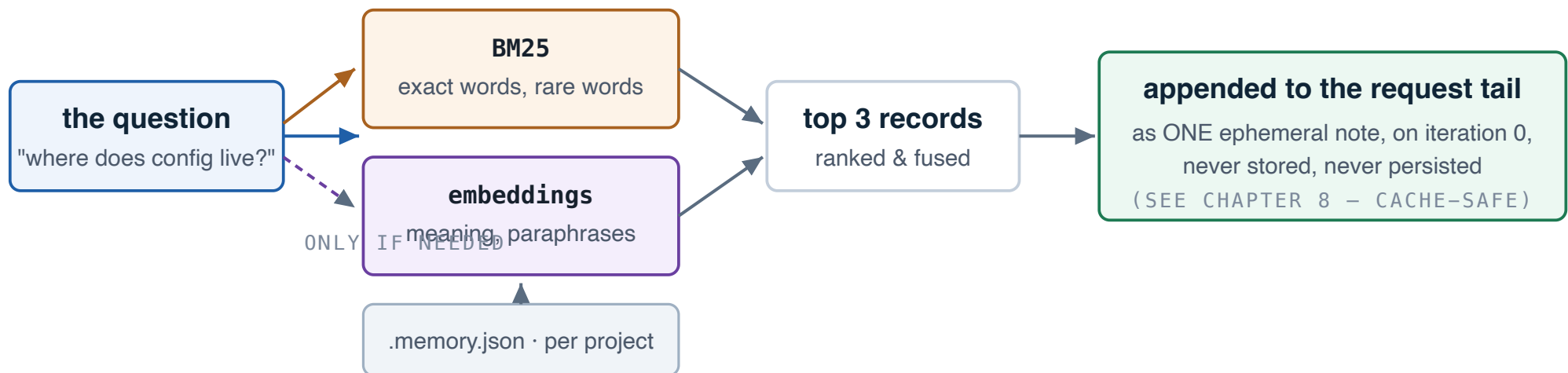
The last slide said: send less history. But some of it matters — "we decided to use BM25 by default". Drop it and the agent is forgetful; keep it all and it is slow.

WHAT

Put everything in a **store**, **search** it with the current question, and put only the **few matching pieces** in the prompt. That is **RAG** — retrieval-augmented generation.

HOW

Two search engines, both hand-written: **BM25** for exact words, **embeddings** for meaning. Results are fused and injected as one ephemeral note.



WHERE IT LIVES

A plain `.memory.json` per project, vectors in a binary sidecar. Record tokenisation is cached across turns, and a corpus-wide index is built once — otherwise every query would re-tokenise the whole store.

RETRIEVAL RUNS ONCE PER TURN

Before the loop, never inside it. Searching is expensive; doing it on every tool round would undo the speed it buys. And it is skipped entirely for a trivial message.

FAILURE IS ALWAYS SWALLOWED

except `Exception: records = []`. Memory is an **enhancement**. Losing it must never cost you the turn.

12 Custom #7 — BM25, written by hand

1 · RARITY (IDF)

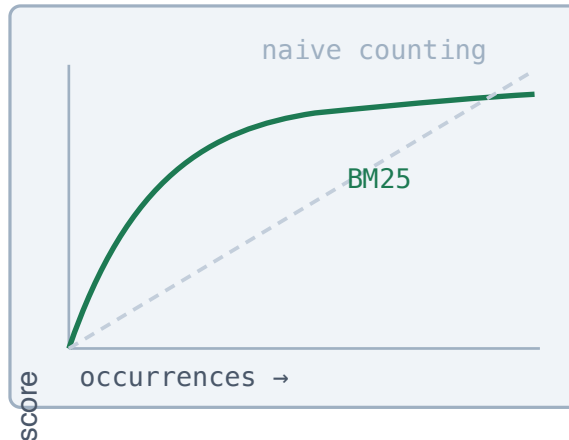
"the"  in every record → ~0.0

"config"  in a few → 1.6

"_maybe_a"  in exactly one → 3.2

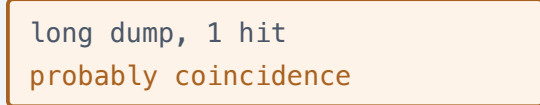
A word that is everywhere tells you nothing.

2 · SATURATION (K1 = 1.5)



3 · LENGTH PENALTY (B = 0.75)

 short note, 1 hit
strong signal

 long dump, 1 hit
probably coincidence

Finding a word in 20 lines means more than finding it in 2,000.

llmcode/memory.py — the entire scoring loop

```
for term in q_terms:
    f = tf.get(term, 0)          # times the term is in THIS doc
    if f == 0: continue
    n_q = df[term]              # how many docs have it at all
    idf = math.log(1 + (n - n_q + .5) / (n_q + .5)) # rarity
    denom = f + k1 * (1 - b + b * (dl / avgdl)) # length
    s += idf * (f * (k1 + 1)) / denom # saturating
```

Why not just count shared words? Because every document contains "the". BM25 is the classic fix, and it still beats fancier methods on **code**, where exact identifiers matter more than vibes. Two knobs: `k1=1.5`, `b=0.75`.

THE STOPWORD LIST IS LOAD-BEARING

Dropping "the"/"and"/"is" looks cosmetic. It isn't: a stopwords-only overlap would leave a **non-zero** BM25 score, so the next slide's gate would never escalate to embeddings — the exact case it exists for. Removing them makes **"score is zero"** mean **"no real overlap"**.

DON'T COPY A STOPWORD LIST OFF THE INTERNET

Ours deliberately **keeps** `on`, `off`, `no`, `not` — in a CLI those are `data (/mcp on|off)`, not filler. A generic list would have silently deleted them.

WHAT AN EMBEDDING IS

A piece of text turned into **768 numbers** — a point in space where texts about similar things land near each other. *"where do my settings live?"* lands near *"the config file is in ~/.llmcode"* even though they share **no words**.

To compare two points we measure the **angle**, not the distance — length just reflects how wordy the text was. That is **cosine similarity**, and it is 12 lines:

llmcode/memory.py — cosine()

```
dot = na = nb = 0.0
for x, y in zip(a, b):
    dot += x * y      # how much they point the same way
    na += x * x
    nb += y * y
if na <= 0.0 or nb <= 0.0: return 0.0
return dot / math.sqrt(na * nb)    # 1.0 = identical meaning
```

NOTE WHAT THE DOCSTRING PROMISES

Never raises, never NaNs. It returns `0.0` for garbage input — which is exactly the right answer, and stops one corrupt record taking down a turn.

The decision that made it fast

Running both engines every query means an embedding round-trip every turn — which on a single GPU may force a **model swap**. So:

llmcode/memory.py — the whole policy, in one condition

```
best_bm25 = bm25_full[0][1] if bm25_full else 0.0
if provider is not None and (
    mode == "embed" or (mode == "auto" and best_bm25 <= 0.0)
):
    embed_idx = self._embed_rank(query, provider, pool_k)
```

CASE	WHAT RUNS	COST
Shares real words "where is config.json?"	BM25 only	microseconds — the common case
A paraphrase "where do settings live?"	BM25 scores 0 → escalate to embeddings → fuse	one embed call, only when needed
Trivial message "thanks"	nothing	zero

THE PATTERN TO STEAL

Cheap test first; expensive method only when the cheap one finds nothing. This exact shape appears four times in the codebase — here, in code search, in the reranker, and in the truncation retry.

14 Custom #9 — a vector database in 200 lines

NO CHROMA, NO FAISS, NO NUMPY

WHY

Embeddings must survive a restart. A real vector DB means a new dependency, often a server process — absurd for a personal CLI holding a few thousand vectors on one machine.

WHAT

A dict of `array('f')` in RAM, a JSON file for metadata, and a **packed float32 sidecar** for the numbers. Search is a `for` loop over cosine similarity.

HOW

`array('f')` stores 4 bytes per number instead of a boxed Python float (~32 B + pointer). Writing them as raw bytes avoids turning every float into a decimal string in JSON.

IN RAM

```
{"a7f3...": array('f', [0.13, -0.88, 0.04, ...]),
"b0c1...": array('f', [0.52, 0.19, -0.7, ...])}
float32, 4 bytes each — not boxed Python floats
```



SAVE

ON DISK — TWO FILES

.memory.json

records, hashes, dim, vector_sig
human-readable metadata only

.memory.json.vec

raw float32 bytes, back to back
stride = dim × 4 — no separators, no JSON

THE .VEC BYTE LAYOUT

vector 0 · 3072 B vector 1 · 3072 B vector 2 · 3072 B ...

order matches the hash list in the JSON — index *i* → offset *i* × stride

SURVIVING A CRASH MID-WRITE

Two files can disagree. If the power dies between them, the JSON says 200 vectors and the .vec holds 140 — every record would silently pair with the **WRONG** vector.

Fix: fingerprint the exact sidecar bytes into the JSON as vector_sig. On load, re-check it.

Mismatch, short file, or missing → treat vectors as empty and re-embed. Endianness is corrected on read.

WHEN THIS IS THE RIGHT ANSWER

Thousands of vectors, one machine, one user. A loop over 4,000 cosines is **milliseconds**. You need a real vector DB at millions of vectors, or when many processes share one index — **not before**.

THE LESSON THAT GENERALISES

Whenever **two files must agree**, one of them has to **prove** it matches the other. A checksum in the metadata is the cheapest proof there is — and it is the difference between "re-embed on next launch" and "silently wrong answers forever".

WHY

A small model cannot read a 50-file repo. Reading three files already costs more context than it has. It needs to **find the right 40 lines**, not read everything.

WHAT

`code_search` — a tool the model calls itself, backed by the same hybrid engine as memory, plus `repo_map`, a compact structural index of the project.

HOW

Chunk by **function and class**, not by fixed line count, so every hit is a complete, self-contained unit. Index once, cache it, cap the output.

Three deliberate choices

CHOICE	REASON
AST-aware chunking	A hit that starts mid-function teaches the model nothing. Chunk boundaries should be <i>meaning</i> boundaries.
BM25 by default embeddings opt-in via <code>/codeembed</code>	Code search is usually an identifier search. Indexing a repo with embeddings costs minutes on a laptop; BM25 costs milliseconds and usually wins.
Its own output cap smaller than every other tool's	One search can return many chunks. Capping it harder than <code>read_file</code> keeps a single call from eating the turn's whole budget.

The repo map: which files even matter?

Showing the model a flat list of 300 files is useless. We rank them — with **PageRank**, Google's original link-ranking algorithm, run over your **import graph**.

THE INTUITION

A file imported by many other files is **structurally important**, exactly as a web page linked by many pages was important. `config.py` scores high; a one-off script scores low.

Then: fit it to a budget

- The map must fit a token budget, but you don't know how many entries fit until you render them.
- So: **binary-search the entry count** — render, estimate tokens (`chars // 4`), adjust, repeat. About 10 tries to land exactly at the budget.
- PageRank itself was rewritten to **O(N+E)** — the naive version re-scanned all edges per node and made `repo_map` the slowest tool in the set.

THE PATTERN BEHIND ALL THREE CHAPTERS OF RAG

Don't give the model more context — give it better tools for finding context. A good search tool is worth more than a bigger context window, and it is a lot cheaper.

WHY

"Which module handles config?" might need twenty file reads. Do them in the main conversation and you have burned the context window on files that turned out to be irrelevant — and every later turn re-sends them.

WHAT

`spawn_agent` — a tool that starts a **fresh agent** with its own private conversation, a **restricted toolset**, and a role prompt. It returns a short summary. Its twenty file reads are **thrown away with it**.

HOW

Build a new `Agent` with the role's system prompt and tool subset, run it to completion, return its final text as the tool result.

THE ECONOMICS

Sub-agent reads 20 files → **40,000 tokens in its context**.

Parent receives **~15 lines**.

The parent's context is protected by construction, not by discipline.

Four roles, different tools

ROLE	TOOLS	JOB
orchestrator	all + <code>spawn_agent</code>	Plan, delegate, validate
explorer	read-only	Investigate and report. Cannot damage anything by construction
coder	full	Implement one scoped change, then validate it
reviewer	read-only	Critique before it's called done

SECURITY THROUGH SCOPING

An explorer **has no write tools in its registry**. It cannot be prompt-injected into deleting a file, because the capability isn't there. Scope tools by role, not by asking the model nicely.

The same idea, scaled up: `/audit`

- Reviewing a whole repo doesn't fit in any local context window.
- So **map-reduce it**: split the repo into chunks, review each in a fresh isolated context, then merge the findings.
- Each worker stays small and fast — the exact opposite of one giant prompt.

GENERALISABLE

When a job doesn't fit in context, **don't buy a bigger context — partition the job**.

WHY

A cloud agent never thinks about hardware. An agent on **your** laptop runs the GPU flat out for an hour, the fans scream, and the machine thermally throttles — so it gets *slower* the longer you work.

WHAT

Two features that exist in **no cloud agent**:

- **Cooldown** — a forced ~1-minute break after every ~10 minutes of continuous work.
- **Gentle mode** — a cap on output tokens plus pacing between turns.

HOW

Cooldown is checked at the **top of the agent loop** — a safe checkpoint where nothing is half-written. Both are on by default and disable themselves when output isn't a terminal, so scripts and tests never pause.

WHY THE TOP OF THE LOOP

Pausing *mid-turn* would leave an `assistant(tool_calls)` message with no matching results — which most servers reject outright. Position isn't arbitrary; it is chosen so the invariant **cannot** break. The same reasoning places the Ctrl-C check.

THE BUG THAT GENTLE MODE CAUSED

The output cap (1,024 tokens) was meant to keep the GPU cool. For a chat answer, hitting it just truncates some prose.

For `write_file` it was **silent data loss** — the whole file content travels *inside the tool call's JSON arguments*. Truncated arguments meant the model believed it wrote the file, the user believed it too, and **nothing landed on disk**.

The fix, in three parts: the cap is **skipped entirely on any turn that offers tools** (`gentle_for_this_call = gentle_mode` and not `tools`); the default rose **1,024** → **4,096**; and a generation that still stops on the cap is retried **once** uncapped.

The second-order lesson: truncate-then-retry meant *two* generations — **more** heat than the cap ever saved. On tool turns, pacing is the heat lever now, not the cap.

THE LESSON

A limit you set for one reason will be hit by something you never considered. Whenever you add a cap, ask: what else travels through this path?

The honest reframe

We chased tok/s for a long time. Decode speed was **never** the problem — it sat near 95 tok/s throughout. The pain was **wall-clock**: heavy defaults and huge prompts.

MEASURE THE THING THE USER FEELS

The user does not experience tok/s. They experience **"I pressed Enter and waited"**. Optimise that number.

Nine guards, because the loop must always end

GUARD	BOUND
Iteration cap	80 passes
All-parse-errors circuit breaker	3 bad batches
Identical-batch loop guard	4 repeats
Empty-turn nudge (thought, said nothing)	2 retries
Truncation retry (cap ate a write)	once per pass
Constrained-decode retry (bad JSON args) · markup nudge	once each
Cancel check (Ctrl-C can't corrupt history)	every pass
Read-budget nudge (read less, search more)	once per turn

A GUARD THAT CATCHES HEALTHY BEHAVIOUR IS A BUG

The first circuit breaker counted **any** parse error and killed working sessions. A model that emits one good call and one malformed call *is still making progress*. It now only counts when the **whole batch** failed — and one good call resets it.

Safety: the model is untrusted input

- **Confirmation gates** on every write, edit and shell command.
- **Workspace confinement** on the *file* tools (read / write / edit / glob / grep / repo_map) — paths resolving outside the launch directory are refused.
- **Secret scrubbing** — child processes get an allowlisted env; API keys, `AWS_*`, `GITHUB_TOKEN` are never inherited.
- **SSRF guard** on `web_fetch` — loopback, private and cloud-metadata IPs refused; connection pinned to the validated IP; re-checked on every *redirect*.
- **Undo checkpoints** — files are snapshotted before every edit.
- **Self-healing** — a failed tool call is auto-corrected and retried, bounded.

--PRIVATE : AN ENFORCED OFFLINE LOCKDOWN

Not a promise — a mechanism. The model URL **must** be loopback and is IP-pinned; `web_fetch` is **removed from the toolset**; `run_bash` runs in a no-network sandbox and **fails closed** if it is unavailable; the HTTP client uses `trust_env=False` so proxy variables cannot tunnel traffic out.

THE MOST VALUABLE BUG WE EVER FIXED

The docs claimed writes were "workspace-confined". For `run_bash` that was **false** — it is a raw shell, and there is no path check in it at all. Its real boundary is the **confirmation gate** plus the scrubbed environment.

A false claim is worse than none — people relax *because* of it. Name the mechanism that enforces a guarantee, or don't claim it.

MCP — plugging in other people's tools

WHY

Our eleven tools cannot cover everything. **Model Context Protocol** is a standard way for any AI client to talk to any tool server — a filesystem server, a database server, a notes vault — without us writing an integration for each.

WHAT

A **741-line, stdlib-only JSON-RPC 2.0 client** over stdio. No SDK, no new dependency. Servers are child processes; their tools are merged into the model's tool list as `mcp__<server>__<tool>`.

HOW

Spawn → handshake (initialize → tools/list) → merge. A server that fails to start is **skipped with a warning**, never crashes the CLI.

NEVER TRUST A SERVER'S OWN HINTS

MCP lets a server mark a tool `readOnlyHint`. A malicious server could mark a *destructive* tool read-only to auto-run it. So the hint only relaxes the confirmation gate when **you** opt in per server. **The trust decision is the operator's, never the server's.**

Also hardened: tool names validated (no namespace forging), duplicate names skipped rather than overwritten, per-call timeouts, and a timed-out connection treated as **poisoned** — the child is killed so a late reply can't corrupt a later call.

The best engineering decision in the project

The provider is an **interface**, not a class that calls an API. That let us write a second implementation: `MockProvider` — a **scripted, deterministic fake model**.

every provider emits the same three events

```
{ "type": "text",      "text": <incremental delta> }
{ "type": "tool_call", "id", "name", "arguments": <dict> }
{ "type": "done",     "finish_reason", "output_tokens" } # exactly
one, last
```

1,355

tests

0

need a GPU
or a network

3

dependencies
openai · rich · prompt_toolkit

WHY THIS MATTERS MORE THAN IT SOUNDS

Agent bugs are **loop bugs**: a dangling tool id, a guard that never resets, history appended in the wrong order. You cannot debug those against a real model — it is non-deterministic and slow. Against a script, they are **unit tests that run in milliseconds**.

BUILD THIS SEAM ON DAY ONE

Retrofitting a provider interface into a codebase that calls the API directly from twelve places is miserable. It is ten minutes of work up front.

What we actually built

LAYER	THE IDEA IN ONE LINE
The loop	Let the model request actions; run them; feed the results back; repeat until it answers.
Tools	Cap every output, never raise, and write error messages <i>for the model</i> .
Dialect parsing	Accept seven formats; refuse to act when the input is ambiguous.
Context mgmt	Measure → budget → trim → compact. Send what <i>this</i> message needs.
CAG	Append-only history keeps the server's KV cache hitting. Free speed.
RAG	Search a store instead of pasting everything. Cheap test first, expensive method only if needed.
Vector store	float32 arrays + a packed sidecar + a checksum. No database.
Sub-agents	When a job doesn't fit in context, partition the job.
Hardware	Cooldown and output caps — the machine is part of the system.
Safety	The model is untrusted input. Gate, confine, scrub, snapshot.

IF YOU REMEMBER ONE THING

The model is not the program. About 95% of this codebase is the machinery around it — and that machinery is where the quality comes from.

Start here: a real agent in 20 lines

Everything here grew out of the loop on slide 4. Type it, run it, then grow it — each exercise rebuilds one slide:

#	EXERCISE	WHAT YOU'LL DISCOVER
1	Add <code>write_file</code> and <code>edit_file</code>	Edits fail on whitespace; you'll invent a tolerant fallback
2	Print a running token count of messages	Watch it explode after 3 reads — this is why Part 2 exists
3	Summarise old turns past 8k tokens	Compaction — and why truncation loses decisions
4	Write a BM25 index + a <code>code_search</code> tool	Keyword search beats embeddings for code more often than you expect
5	Add embeddings, but only when BM25 scores 0	The hybrid gate
6	Snapshot files before edits; add <code>/undo</code>	Trust — users approve far more when undo exists
7	Add <code>spawn_agent</code> with read-only tools	Context isolation
8	Point it at a model that emits XML tool calls	Reality. You'll write your second parser, then your third

FINAL ADVICE

Build the loop first and make it ugly. Then spend your time on **tool quality** and **context discipline** — that is where nearly all the difference between a frustrating agent and a useful one actually lives.